

""

=====
=====

THE VOID – ADGINUS Core Intelligence Engine
Version: 1.3.0 – OpenAI + Voyage Semantic
Memory Build

"I remember. I recognize. I learn. I dream.
I KNOW."

Single-file build merging all 9 modules:

- EmbeddingBridge (Voyage AI – real semantic memory search)
- SuperMemory (now embedding-aware, falls back to keyword search)
- PatternRecognitionEngine
- TurboLearnEngine
- DreamMode + InstinctEngine
- VoidPersistence (atomic writes, thread-safe, SAFE key parsing –
no more eval() on loaded save files)
- LLMBridge (OpenAI Chat Completions API, proxy-safe, rate-limited, offline fallback)
- BackgroundCycle (thread-safe with locks)
- TheVoid Orchestrator (knowledge-matching in think() actually works
now; dream() actually calls the LLM bridge; depth actually changes behavior)

Requires (set as environment variables, or in Bolt's env settings – never hardcode these into the file):

- OPENAI_API_KEY – or pass openai_api_key in config
- VOYAGE_API_KEY – or pass voyage_api_key in config (optional – without it, memory search still works via keyword matching, just not semantic matching)

Usage:

```
from adginus_void import TheVoid
void = TheVoid(name="ADGINUS-PRIME",
config={
    "model": "gpt-4o-mini",          #
set to whatever OpenAI model you're using
    "embedding_model": "voyage-3.5"
})
void.boot()
void.learn("python", "Python is
versatile", domain="programming")
void.perceive("AI can compose music",
source="news", intensity=0.7)
thought = void.think("How to combine AI
and creativity?")
void.save()

=====
=====
"""
import ast
import hashlib
import json
```

```

import os
import re
import random
import threading
import time
import tempfile
import urllib.request
import urllib.error
from collections import defaultdict, deque,
Counter
from datetime import datetime, timedelta
from typing import Dict, List, Optional,
Any, Callable

#
=====
=====
# 0. EMBEDDING BRIDGE – real semantic
search, with graceful fallback
#
=====
=====
def cosine_similarity(vec_a: List[float],
vec_b: List[float]) -> float:
    """Plain-Python cosine similarity – no
numpy dependency needed."""
    if not vec_a or not vec_b or len(vec_a)
!= len(vec_b):
        return 0.0
    dot = sum(a * b for a, b in zip(vec_a,
vec_b))
    norm_a = sum(a * a for a in vec_a) **
0.5
    norm_b = sum(b * b for b in vec_b) **

```

```

0.5
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot / (norm_a * norm_b)

class EmbeddingBridge:
    """
    Bridges SuperMemory to a real
    embeddings API (Voyage AI by default –
    the standard pairing with Claude, since
    Anthropic doesn't run its own
    embeddings endpoint). Same shape as
    LLMBridge: works directly against
    the provider, or through your own proxy
    if you'd rather not put a raw
    key in the app.

    If no key/proxy is configured, or a
    call fails for any reason,
    everything that uses this bridge falls
    back to the existing keyword
    search automatically – nothing breaks,
    it just doesn't get the
    semantic upgrade until a key is added.
    """

    VOYAGE_DEFAULT_URL =
    "https://api.voyageai.com/v1/embeddings"

    def __init__(self, api_key: str = None,
model: str = "voyage-3.5",
                base_url: str = None,
max_calls_per_hour: int = 60):

```

```

        self.model = model
        self.api_key = api_key or
os.environ.get("VOYAGE_API_KEY")
        self.base_url = base_url or
self.VOYAGE_DEFAULT_URL
        self.using_proxy = bool(base_url)
and "voyageai.com" not in self.base_url
        self.call_count = 0
        self.total_texts_embedded = 0
        self._call_timestamps = []
        self.max_calls_per_hour =
max_calls_per_hour
        self._lock = threading.RLock()

    def is_available(self) -> bool:
        if not self.base_url:
            return False
        if self.using_proxy:
            return True
        return bool(self.api_key)

    def _check_rate_limit(self) -> bool:
        now = time.time()
        self._call_timestamps = [t for t in
self._call_timestamps if now - t < 3600]
        return len(self._call_timestamps) <
self.max_calls_per_hour

    def embed(self, texts: List[str]) ->
Optional[List[Optional[List[float]]]]:
        """
        Embed a batch of texts. Returns a
list of vectors (same order/length
as `texts`), or None entries for

```

```

any text that failed. Returns None
entirely if unavailable/rate-
limited/network error – callers should
treat that as "fall back to keyword
search," not as an exception.
"""
    with self._lock:
        if not texts:
            return []
        if not self.is_available() or
not self._check_rate_limit():
            return None

        headers = {"Content-Type":
"application/json"}
        if not self.using_proxy:
            headers["Authorization"] =
f"Bearer {self.api_key}"

        payload = {"input": texts,
"model": self.model}

        try:
            req =
urllib.request.Request(
                self.base_url,
                data=json.dumps(payload).encode(),
                headers=headers,
                method="POST"
            )
            with
urllib.request.urlopen(req, timeout=20) as
resp:
                data =

```

```

json.loads(resp.read().decode())

        items =
sorted(data.get("data", []), key=lambda x:
x.get("index", 0))
        vectors =
[item.get("embedding") for item in items]

        self.call_count += 1

self._call_timestamps.append(time.time())
        self.total_texts_embedded
+= len(texts)

        # Defensive: if the API
returned a different count than we
        # sent, pad with None
rather than silently misaligning
        # embeddings with the wrong
memories.
        while len(vectors) <
len(texts):
            vectors.append(None)
        return vectors[:len(texts)]

    except Exception:
        return None

    def embed_one(self, text: str) ->
Optional[List[float]]:
        result = self.embed([text])
        if result and result[0] is not
None:
            return result[0]

```

```

        return None

    def get_stats(self) -> Dict:
        with self._lock:
            return {
                "available":
self.is_available(),
                "using_proxy":
self.using_proxy,
                "model": self.model,
                "total_calls":
self.call_count,
                "total_texts_embedded":
self.total_texts_embedded,
                "calls_in_last_hour":
len(self._call_timestamps),
                "max_calls_per_hour":
self.max_calls_per_hour
            }

class SuperMemory:
    """
    Multi-layered memory:
    - STM: deque recent fast-access
    - LTM: dict persistent indexed
    - EM: high significance
    - Muscle: repeated patterns auto-
execute
    """
    def __init__(self, stm_capacity: int =
100):
        self.short_term =

```

```

deque(maxlen=stm_capacity)
        self.long_term: Dict[str, Dict] =
    {}
        self.emotional: Dict[str, float] =
    {}
        self.muscle_memory: Dict[str, Dict]
= {}
        self.memory_index: Dict[str, set] =
defaultdict(set)
        self.access_count: Dict[str, int] =
defaultdict(int)
        self.creation_log: List[tuple] = []
        self.total_memories = 0
        self._lock = threading.RLock()
        # Optional EmbeddingBridge,
attached post-construction by TheVoid.
        # When present and available,
store()/recall() use real semantic
        # similarity. When absent (the
default), everything behaves
        # exactly as the keyword-based
version did – no behavior change.
        self.embedder:
Optional["EmbeddingBridge"] = None

    def _generate_hash(self, content: str)
-> str:
        base = f"{content}|
{self.total_memories}|
{datetime.now().timestamp()}"
        return
hashlib.sha256(base.encode()).hexdigest()
[:16]

```

```

    def _tokenize(self, text: str) ->
List[str]:
        # cleaner tokenizer, strips
punctuation, lowercases
        return [w for w in
re.findall(r"\b\w{2,}\b", text.lower())]

    def store(self, content: str, tags:
List[str] = None, significance: float =
0.5, source: str = "input") -> str:
        with self._lock:
            mem_hash =
self._generate_hash(content)
            timestamp =
datetime.now().isoformat()
            memory_node = {
                "hash": mem_hash,
                "content": content,
                "tags": tags or [],
                "significance":
float(max(0.0, min(1.0, significance))),
                "source": source,
                "timestamp": timestamp,
                "access_count": 0,
                "connections": [],
                "decay_rate": max(0.01, 1.0
- significance),
                "strength": significance,
                "evolved": False,
                "last_accessed": timestamp
            }

self.short_term.append(mem_hash)
self.long_term[mem_hash] =

```

```

memory_node
    if significance >= 0.7:
        self.emotional[mem_hash] =
significance

memory_node["emotional_flag"] = True

        for tag in (tags or []):

self.memory_index[tag.lower()].add(mem_hash
)
        for word in
self._tokenize(content):
            if len(word) > 3:

self.memory_index[word].add(mem_hash)


self.creation_log.append((timestamp,
mem_hash))
        self.total_memories += 1

self._check_muscle_memory(content,
mem_hash)


        # Semantic upgrade: if an
embedder is attached and reachable,
        # embed this memory now so
recall() can find it by meaning
        # later, not just by shared
words. Any failure here is
        # swallowed on purpose – a
memory always gets stored even if
        # the embedding call fails; it

```

```

just won't be semantically
    # searchable until
backfill_embeddings() catches it up.
    if self.embedder and
self.embedder.is_available():
        try:
            vec =
self.embedder.embed_one(content)
            if vec:

memory_node["embedding"] = vec
        except Exception:
            pass

    return mem_hash

    def _check_muscle_memory(self, content:
str, mem_hash: str):
        # If similar content seen 3+ times,
promote to muscle memory
        key = content.lower()[:80]
        if key not in self.muscle_memory:
            self.muscle_memory[key] =
{"count": 0, "examples": []}
            self.muscle_memory[key]["count"] +=
1
            self.muscle_memory[key]
["examples"].append(mem_hash)
            if self.muscle_memory[key]["count"]
>= 3:
                self.muscle_memory[key]["auto"]
= True

    def recall(self, query: str,

```

```

max_results: int = 10, semantic_hook:
Callable = None) -> List[Dict]:
    """
        semantic_hook: optional
callable(query, candidates) -> scored list,
        for a fully custom re-ranking step
layered on top of everything else.

        If self.embedder is attached and
available, this does real semantic
        search: the query is embedded and
compared by cosine similarity
        against every long-term memory's
embedding (not just ones sharing a
        literal keyword) – so "python" can
now find a memory about
        "programming languages" even with
zero words in common. Memories
        that don't have an embedding yet
(e.g. stored before an embedder was
        attached) fall back to keyword
scoring for just that memory, so
        nothing is silently excluded.

        If no embedder is
attached/available, behavior is identical
to the
        original keyword-only
implementation.
    """
    with self._lock:
        query_words =
self._tokenize(query)
        now = datetime.now()

```

```

        def _recency(node) -> float:
            try:
                last =
datetime.fromisoformat(node.get("last_acces
sed", node["timestamp"]))
                hours_old = (now -
last).total_seconds() / 3600
                return 1.0 / (1.0 +
hours_old / 24.0)
            except Exception:
                return 0.5

        def _keyword_score(node) ->
float:
            overlap =
len(set(query_words) &
set(self._tokenize(node["content"]))) /
max(len(query_words), 1)
            return (overlap * 0.5 +
node["significance"] * 0.3 +

min(node["access_count"] / 10.0, 1.0) * 0.1
+ _recency(node) * 0.1)

        use_semantic =
bool(self.embedder and
self.embedder.is_available())
        query_vec =
self.embedder.embed_one(query) if
use_semantic else None
        # If embedding the query itself
fails, quietly drop back to
        # keyword mode rather than

```

```

returning nothing.
        use_semantic = use_semantic and
query_vec is not None

        scored = []
        if use_semantic:
            # Real semantic search:
            compare against every memory that
            # has an embedding, not
            just keyword-index candidates –
            # that's the whole point
            (find related memories that don't
            # share literal words).
            for node in
self.long_term.values():
                node_vec =
node.get("embedding")
                if node_vec:
                    sim =
cosine_similarity(query_vec, node_vec)
                    score = (sim * 0.7
+ node["significance"] * 0.15 +

min(node["access_count"] / 10.0, 1.0) *
0.05 + _recency(node) * 0.1)
                else:
                    # No embedding on
this one yet – don't drop it,
                    # just fall back to
keyword scoring for it alone.
                    score =
_keyword_score(node) * 0.6 # slight
penalty vs. semantically-scored peers
                    scored.append((score,

```

```

node))
        else:
            candidates = set()
            for word in query_words:
                if word in
self.memory_index:

candidates.update(self.memory_index[word])
                if not candidates:
                    candidates =
set(list(self.short_term)[-20:])
                for h in candidates:
                    node =
self.long_term.get(h)
                    if node:

scored.append((_keyword_score(node), node))

                scored.sort(key=lambda x: x[0],
reverse=True)

                if semantic_hook and scored:
                    try:
                        scored =
semantic_hook(query, scored)
                    except Exception:
                        pass

                results = []
                for score, node in
scored[:max_results]:
                    node["access_count"] += 1
                    node["last_accessed"] =
datetime.now().isoformat()

```

```

self.access_count[node["hash"]] += 1
        results.append(**node,
            "_score": round(score, 3))
        return results

    def backfill_embeddings(self,
        batch_size: int = 50) -> Dict:
        """
        Embed any long-term memories that
        don't have an embedding yet –
        useful right after attaching an
        embedder to a memory store that
        already has history (e.g. loaded
        from a save file made before
        embeddings existed, or memories
        stored while the embedder was
        temporarily unavailable). Batches
        calls to be efficient.
        """
        if not self.embedder or not
        self.embedder.is_available():
            return {"status":
                "unavailable", "backfilled": 0}

        with self._lock:
            missing = [(h, n) for h, n in
                self.long_term.items() if not
                n.get("embedding")]
            backfilled = 0
            for i in range(0, len(missing),
                batch_size):
                batch = missing[i:i +
                    batch_size]

```

```

        texts = [n["content"] for
_, n in batch]
        vectors =
self.embedder.embed(texts)
        if not vectors:
            continue # provider
unavailable/rate-limited mid-run – stop
trying, keep what we have
        for (h, node), vec in
zip(batch, vectors):
            if vec:
                node["embedding"] =
vec
                backfilled += 1

        return {"status": "done",
"backfilled": backfilled, "still_missing":
len(missing) - backfilled}

    def apply_decay(self, hours_threshold:
int = 72) -> Dict:
        """Actual forgetting
implementation"""
        with self._lock:
            now = datetime.now()
            pruned = 0
            strengthened = 0
            for h, node in
list(self.long_term.items()):
                if
node.get("emotional_flag"):
                    continue # never decay
emotional
                try:

```

```

        last =
datetime.fromisoformat(node.get("last_accessed", node["timestamp"]))
        hours = (now -
last).total_seconds() / 3600
        if hours >
hours_threshold:
            # decay strength
            node["strength"] =
max(0.01, node["strength"] * (1.0 -
node["decay_rate"] * 0.1))
            if node["strength"]
< 0.05 and node["access_count"] < 2:
                # prune weak,
unused
                del
self.long_term[h]
                pruned += 1
            else:
                strengthened +=
1
        except Exception:
            continue
        return {"pruned": pruned,
"decayed": strengthened}

    def connect_memories(self, hash1: str,
hash2: str, relation: str = "associated"):
        with self._lock:
            if hash1 in self.long_term and
hash2 in self.long_term:
                self.long_term[hash1]
["connections"].append({"to": hash2,
"relation": relation})

```

```

        self.long_term[hash2]
["connections"].append({"to": hash1,
"relation": relation})

    def search_emotional(self) ->
List[Dict]:
        with self._lock:
            results = [self.long_term[h]
for h in self.emotional if h in
self.long_term]
            results.sort(key=lambda x:
x["significance"], reverse=True)
            return results

    def get_stats(self) -> Dict:
        with self._lock:
            strongest = None
            if self.long_term:
                strongest =
max(self.long_term.values(), key=lambda x:
x["significance"])
                strongest = {"hash":
strongest["hash"], "sig":
strongest["significance"], "content":
strongest["content"][:80]}
            return {
                "total_memories":
self.total_memories,
                "stm_size":
len(self.short_term),
                "ltm_size":
len(self.long_term),
                "emotional_memories":
len(self.emotional),

```

```

        "muscle_memory_patterns":
len(self.muscle_memory),
        "indexed_keywords":
len(self.memory_index),
        "strongest_memory":
strongest,
        "total_connections":
sum(len(n["connections"]) for n in
self.long_term.values())
    }

    def export_memories(self) -> str:
        with self._lock:
            return json.dumps({
                "total_memories":
self.total_memories,
                "long_term":
self.long_term,
                "emotional":
self.emotional,
                "muscle_memory":
self.muscle_memory,
                "creation_log":
self.creation_log
            }, indent=2, default=str)

#
=====
=====
# 2. PATTERN RECOGNITION ENGINE
#
=====
=====
class PatternRecognitionEngine:

```

```

def __init__(self):
    self.observed_sequences = []
    self.known_patterns = {}
    self.anomalies = []
    self.correlations = {} # pair_key
(sorted tuple) -> correlation dict; was
defaultdict(list)
    self.frequency_map = Counter()
    self.co_occurrence =
defaultdict(Counter)
    self.trend_data = defaultdict(list)
    self.trend_directions = {}
    self.templates = {}
    self.template_matches =
defaultdict(int)
    self.total_observations = 0
    self.patterns_detected = 0
    self.anomalies_detected = 0
    self.sensitivity = 0.5
    self.min_pattern_length = 2
    self.max_pattern_length = 50
    self._lock = threading.RLock()

    def _extract_elements(self, data) ->
List[str]:
        if isinstance(data, str):
            tokens = re.findall(r'\b\w+\b',
data.lower())
            return [t for t in tokens if
len(t) > 1]
        elif isinstance(data, (int,
float)):
            return [str(data)]
        elif isinstance(data, list):

```

```

        result = []
        for item in data:

result.extend(self._extract_elements(item))
        return result
        elif isinstance(data, dict):
            result = []
            for k, v in data.items():

result.append(str(k).lower())

result.extend(self._extract_elements(v))
            return result
        else:
            return [str(data).lower()]

    def observe(self, data, context: str =
"general", timestamp: str = None) -> Dict:
        with self._lock:
            timestamp = timestamp or
datetime.now().isoformat()
            self.total_observations += 1
            elements =
self._extract_elements(data)
            for elem in elements:
                self.frequency_map[elem] +=
1
            for i, elem1 in
enumerate(elements):
                for elem2 in
elements[i+1:]:

self.co_occurrence[elem1][elem2] += 1

```

```

self.co_occurrence[elem2][elem1] += 1

        observation = {
            "data": str(data)[:500],
            "elements": elements[:50],
            "context": context,
            "timestamp": timestamp,
            "observation_id":
self.total_observations
        }

self.observed_sequences.append(observation)
    if len(self.observed_sequences)
> 1000:
        self.observed_sequences =
self.observed_sequences[-500:]

        report = {
            "observation_id":
self.total_observations,
            "elements_found":
len(elements),
            "sequences":
self._detect_sequences(elements),
            "anomalies":
self._detect_anomalies(elements, context),
            "frequency_hits":
self._check_frequency_spikes(elements),
            "correlations":
self._find_correlations(elements),
            "trend_update":
self._update_trends(elements, context,
timestamp)
        }

```

```

        return report

    def _detect_sequences(self, elements):
        # Simple n-gram sequence detection
        seqs = []
        for n in
range(self.min_pattern_length, min(4,
len(elements)+1)):
            for i in range(len(elements)-
n+1):
                seq =
tuple(elements[i:i+n])
                if seq not in
self.known_patterns:
                    # check if seen before
                    count = sum(1 for obs
in self.observed_sequences if
self._contains_seq(obs["elements"], seq))
                    if count >= 3:

self.known_patterns[seq] = {"sequence":
seq, "occurrences": count, "first_seen":
datetime.now().isoformat(), "confidence":
min(1.0, count/10.0)}

self.patterns_detected += 1

seqs.append(self.known_patterns[seq])
        return seqs

    def _contains_seq(self, elements, seq):
        # check if seq appears in elements
        for i in range(len(elements)-
len(seq)+1):

```

```

        if
tuple(elements[i:i+len(seq)]) == seq:
            return True
        return False

    def _detect_anomalies(self, elements,
context):
        anomalies = []
        # anomaly if element never seen
before and sensitivity high
        for elem in elements:
            if self.frequency_map[elem] ==
1 and self.total_observations > 10:
                if random.random() <
self.sensitivity:
                    a = {"type":
"novel_element", "element": elem,
"context": context, "severity": 0.5,
"timestamp": datetime.now().isoformat()}

self.anomalies.append(a)
                    self.anomalies_detected
+= 1
                    anomalies.append(a)
        return anomalies

    def _check_frequency_spikes(self,
elements):
        hits = []
        for elem in elements:
            if self.frequency_map[elem] >
10 and self.frequency_map[elem] % 10 == 0:
                hits.append({"element":
elem, "count": self.frequency_map[elem]})

```

```

        return hits

    def _find_correlations(self, elements):
        # FIX: this used to do
        self.correlations[elem].append(c) – with
        # correlations as
        defaultdict(list), every single observe()
        call
        # where the same two elements co-
        occurred again appended ANOTHER
        # near-duplicate entry, forever,
        for the entire lifetime of the
        # process. Any two words that show
        up together regularly (which
        # is exactly the case this is meant
        to detect) would grow that
        # list without bound. Now keyed by
        the sorted pair itself, so
        # each pair has exactly one entry
        that gets updated in place.
        corrs = []
        for elem in elements:
            top_co =
            self.co_occurrence[elem].most_common(3)
            for other, count in top_co:
                if count >= 3:
                    strength = count /
                    max(self.frequency_map[elem], 1)
                    if strength > 0.3:
                        c = {"pair": (elem,
                        other), "strength": round(strength, 3),
                        "count": count}
                        pair_key =
                        tuple(sorted([elem, other]))

```

```

self.correlations[pair_key] = c
        corrs.append(c)
    return corrs[:5]

    def _update_trends(self, elements,
context, timestamp):
        for elem in set(elements):

self.trend_data[elem].append((timestamp,
context))
            if len(self.trend_data[elem]) >
20:
                self.trend_data[elem] =
self.trend_data[elem][-20:]
                # simple trend direction:
increasing if recent frequency > older
                if len(self.trend_data[elem])
>= 5:
                    self.trend_directions[elem]
= "rising" if len(self.trend_data[elem]) >
10 else "stable"
                    return {"tracked":
len(self.trend_data)}

    def get_stats(self):
        with self._lock:
            return {
                "total_observations":
self.total_observations,
                "unique_elements":
len(self.frequency_map),
                "patterns_detected":
self.patterns_detected,

```

```

        "anomalies_detected":
self.anomalies_detected,
        "templates_defined":
len(self.templates),
        "correlation_pairs":
len(self.correlations),
        "trending_elements":
len(self.trend_directions),
        "sensitivity":
self.sensitivity
    }

#
=====
=====
# 3. TURBO LEARN ENGINE
#
=====
=====
class TurboLearnEngine:
    def __init__(self):
        self.knowledge = {}
        self.domains = defaultdict(dict)
        self.mastery_scores = {}
        self.total_learned = 0
        self.knowledge_gaps = []
        self.skill_trees = {}
        self.learning_rate = 0.15 #
increased from 0.1, now actually used
        self.momentum = 0.0
        self.streak_tracker =
defaultdict(int)
        self.practice_log =
defaultdict(list)

```

```

        self.total_practice = 0
        self.reward_history = []
        self.penalty_history = []
        self.total_rewards = 0
        self.total_penalties = 0
        self.feedback_loop =
defaultdict(list)
        self.domain_bridges =
defaultdict(list)
        self.learning_queue = []
        self._lock = threading.RLock()

    def _categorize_knowledge(self,
content, domain):
        # lightweight categorization
        words = re.findall(r'\b\w+\b',
content.lower())
        return list(set([w for w in words
if len(w) > 5][:5]))

    def absorb(self, topic: str, content:
str, domain: str = "general", difficulty:
float = 0.5, prerequisites: List[str] =
None) -> Dict:
        with self._lock:
            timestamp =
datetime.now().isoformat()
            key = topic.lower()
            prerequisites = prerequisites
or []

            # check gaps
            missing = [p for p in
prerequisites if p.lower() not in
self.knowledge]

```

```

        if missing:

self.knowledge_gaps.append({"topic": topic,
"missing": missing, "timestamp":
timestamp})

        knowledge_node = {
            "topic": topic,
            "content": content,
            "domain": domain,
            "difficulty": difficulty,
            "prerequisites":
prerequisites,
            "absorbed_at": timestamp,
            "times_reviewed": 0,
            "mastery": 0.1,
            "connections": [],
            "applications": [],
            "last_accessed": timestamp,
            "categories":
self._categorize_knowledge(content, domain)
        }
        self.knowledge[key] =
knowledge_node
        self.domains[domain][key] =
knowledge_node
        self.mastery_scores[key] = 0.1
        self.total_learned += 1

        # build skill tree node
        if domain not in
self.skill_trees:
            self.skill_trees[domain] =
{"nodes": {}, "edges": [], "depth": 0}

```

```

        self.skill_trees[domain]
        ["nodes"][key] = {"topic": topic,
                          "difficulty": difficulty, "prerequisites":
prerequisites, "mastery": 0.1}
        for pre in prerequisites:
            self.skill_trees[domain]
        ["edges"].append((pre.lower(), key))

        return {"status": "absorbed",
                "topic": topic, "gaps": missing, "mastery":
0.1}

    def practice(self, topic: str,
performance: float = 0.5, notes: str = "")
-> Dict:
        with self._lock:
            key = topic.lower()
            if key not in self.knowledge:
                return {"status":
"unknown_topic", "topic": topic}
            node = self.knowledge[key]
            # === FIX: learning_rate and
momentum now actually affect mastery ===
            # momentum builds with streaks
            if performance >= 0.7:
                self.streak_tracker[key] +=
1
                self.momentum = min(1.0,
self.momentum + 0.05)
            else:
                self.streak_tracker[key] =
0
                self.momentum = max(0.0,
self.momentum - 0.05)

```

```

        # mastery update uses
        learning_rate + momentum bonus
        delta = (performance -
        node["mastery"]) * self.learning_rate *
        (1.0 + self.momentum * 0.5)
        node["mastery"] = max(0.0,
        min(1.0, node["mastery"] + delta))
        self.mastery_scores[key] =
        node["mastery"]
        node["times_reviewed"] += 1
        node["last_accessed"] =
        datetime.now().isoformat()

self.practice_log[key].append({"performance
": performance, "timestamp":
datetime.now().isoformat(), "notes":
notes})

        self.total_practice += 1

        if performance >= 0.8:

self.reward_history.append({"topic": topic,
"performance": performance})
            self.total_rewards += 1
            elif performance < 0.4:

self.penalty_history.append({"topic":
topic, "performance": performance})
            self.total_penalties += 1

        return {"status": "practiced",
"topic": topic, "new_mastery":

```

```

round(node["mastery"],3), "streak":
self.streak_tracker[key], "momentum":
round(self.momentum,3)}

    def reinforce(self, topic: str,
feedback: str, score: float):
    with self._lock:
        key = topic.lower()

self.feedback_loop[key].append({"feedback":
feedback, "score": score, "timestamp":
datetime.now().isoformat()})

    def generate_curriculum(self, domain:
str = None, limit: int = 10) -> List[Dict]:
    with self._lock:
        candidates = []
        pool = self.knowledge.values()
if not domain else self.domains.get(domain,
{}).values()
        for node in pool:
            # prioritize low mastery,
high prerequisites met, not recently
accessed

            mastery = node["mastery"]
            priority = (1.0 - mastery)
* (1.0 - node["difficulty"]*0.3)
            # check prereqs met
            prereq_mastery = 0
            if node["prerequisites"]:
                scores =
[self.mastery_scores.get(p.lower(), 0) for
p in node["prerequisites"]]
                prereq_mastery =

```

```

sum(scores)/len(scores) if scores else 1
        else:
            prereq_mastery = 1
            priority *= (0.5 +
prereq_mastery*0.5)

candidates.append((priority, node))
        candidates.sort(key=lambda x:
x[0], reverse=True)
            curriculum = [{"topic":
n["topic"], "mastery": n["mastery"],
"priority": round(p,3), "domain":
n["domain"]} for p,n in candidates[:limit]]
            self.learning_queue =
curriculum
            return curriculum

    def transfer_knowledge(self,
from_domain: str, to_domain: str) -> Dict:
        with self._lock:
            if from_domain not in
self.domains or to_domain not in
self.domains:
                return {"status":
"domain_missing"}
                bridges = []
                for topic_key, node in
self.domains[from_domain].items():
                    # if categories overlap,
create bridge
                    for other_key, other_node
in self.domains[to_domain].items():
                        overlap =
set(node.get("categories",[])) &

```

```

set(other_node.get("categories",[]))
        if overlap:
            bridge = {"from":
topic_key, "to": other_key, "overlap":
list(overlap), "strength":
len(overlap)/5.0}

self.domain_bridges[f"{from_domain}->
{to_domain}"].append(bridge)

bridges.append(bridge)
        return {"status": "bridged",
"bridges": bridges, "count": len(bridges)}

    def mastery_decay(self, days: int = 7)
-> Dict:
        with self._lock:
            decayed = 0
            now = datetime.now()
            for key, node in
self.knowledge.items():
                try:
                    last =
datetime.fromisoformat(node["last_accessed"
])
                    delta_days = (now -
last).days
                    if delta_days > days:
                        decay = 0.02 *
(delta_days / days)
                        node["mastery"] =
max(0.0, node["mastery"] - decay)

self.mastery_scores[key] = node["mastery"]

```

```

        decayed += 1
    except:
        continue
    return {"decayed_topics":
decayed}

    def get_mastery_report(self, domain:
str = None) -> Dict:
        with self._lock:
            topics = self.knowledge if not
domain else self.domains.get(domain, {})
            if not topics:
                return {"status":
"no_topics"}
            mastery_values = [n["mastery"]
for n in topics.values()]
            return {
                "domain": domain or "all",
                "total_topics":
len(topics),
                "avg_mastery":
round(sum(mastery_values)/len(mastery_value
s),3) if mastery_values else 0,
                "highest_mastery":
max(topics.items(), key=lambda x: x[1]
["mastery"])[0] if topics else None,
                "lowest_mastery":
min(topics.items(), key=lambda x: x[1]
["mastery"])[0] if topics else None,
                "mastered_topics": sum(1
for m in mastery_values if m >= 0.8),
                "needs_work": sum(1 for m
in mastery_values if m < 0.3),
            }

```

```

        def get_stats(self):
            with self._lock:
                return {
                    "total_knowledge":
self.total_learned,
                    "total_practice_sessions":
self.total_practice,
                    "domains":
list(self.domains.keys()),
                    "skill_trees":
list(self.skill_trees.keys()),
                    "learning_rate":
round(self.learning_rate,4),
                    "momentum":
round(self.momentum,3),
                    "total_rewards":
self.total_rewards,
                    "total_penalties":
self.total_penalties,
                    "knowledge_gaps":
len(self.knowledge_gaps),
                    "domain_bridges":
sum(len(v) for v in
self.domain_bridges.values()),
                    "curriculum_size":
len(self.learning_queue)
                }

#
=====
=====
# 4. DREAM MODE + INSTINCT ENGINE
#

```

```

=====
=====
class DreamMode:
    def __init__(self):
        self.is_dreaming = False
        self.dream_log = []
        self.dream_count = 0
        self.insights = []
        self.hypotheses = []
        self.creative_sparks = []
        self.consolidation_log = []
        self.connections_made = 0
        self.memories_strengthened = 0
        self.memories_pruned = 0
        self.dream_depth = 3
        self.creativity_factor = 0.7
        self.chaos_level = 0.3
        self.recurring_themes = Counter()
        self.dream_symbols = {}
        self._lock = threading.RLock()

    def enter_dream(self,
memory_system=None, pattern_engine=None,
learn_engine=None, duration: int = 100) ->
Dict:
        with self._lock:
            self.is_dreaming = True
            self.dream_count += 1
            dream_session = {
                "dream_id":
self.dream_count,
                "started":
datetime.now().isoformat(),
                "cycles": duration,

```

```

        "insights": [],
        "connections": [],
        "hypotheses": [],
        "consolidations": [],
        "creative_sparks": []
    }
    for cycle in range(duration):
        if memory_system:
            cons =
self._replay_memories(memory_system, cycle)
            if cons:

dream_session["consolidations"].append(cons
)

            if pattern_engine:
                insight =
self._dream_weave_patterns(pattern_engine,
cycle)

                if insight:

dream_session["insights"].append(insight)
                if learn_engine:
                    hypo =
self._synthesize_knowledge(learn_engine,
cycle)

                    if hypo:

dream_session["hypotheses"].append(hypo)
                    if random.random() <
self.chaos_level:
                        spark =
self._creative_chaos(memory_system,
pattern_engine, learn_engine)
                        if spark:

```

```
dream_session["creative_sparks"].append(spark)

        self.is_dreaming = False
        dream_session["ended"] =
datetime.now().isoformat()

self.dream_log.append(dream_session)
    # aggregate

self.insights.extend(dream_session["insights"])

self.hypotheses.extend(dream_session["hypotheses"])

self.creative_sparks.extend(dream_session["creative_sparks"])
    return {
        "dream_id":
self.dream_count,
        "cycles_completed":
duration,
        "insights_generated":
len(dream_session["insights"]),
        "connections_made":
len(dream_session["connections"]),
        "hypotheses_formed":
len(dream_session["hypotheses"]),
        "creative_sparks":
len(dream_session["creative_sparks"]),
        "consolidations":
len(dream_session["consolidations"])
```

```

    }

    def _replay_memories(self,
memory_system, cycle):
        if not memory_system.long_term:
            return None
        # pick two random memories to
connect
        hashes =
list(memory_system.long_term.keys())
        if len(hashes) < 2:
            return None
        h1, h2 = random.sample(hashes, 2)
        n1 = memory_system.long_term[h1]
        n2 = memory_system.long_term[h2]
        # if they share a word, strengthen
both
        words1 = set(re.findall(r'\b\w+\b',
n1["content"].lower()))
        words2 = set(re.findall(r'\b\w+\b',
n2["content"].lower()))
        overlap = words1 & words2
        if overlap:
            n1["strength"] = min(1.0,
n1["strength"] + 0.05)
            n2["strength"] = min(1.0,
n2["strength"] + 0.05)
            self.memories_strengthened += 2

        self.recurring_themes.update(overlap)
        return {"type": "strengthened",
"hashes": [h1, h2], "overlap":
list(overlap)[:3]}
        return None

```

```

        def _dream_weave_patterns(self,
pattern_engine, cycle):
            if not
pattern_engine.known_patterns:
                return None
            # pick a random pattern and
generate insight
            pat =
random.choice(list(pattern_engine.known_pat
terns.values()))
            insight = {"pattern":
pat["sequence"], "confidence":
pat["confidence"], "cycle": cycle,
"timestamp": datetime.now().isoformat(),
"text": f"Dream noticed pattern
{pat['sequence']} with confidence
{pat['confidence']}"}
            self.connections_made += 1
            return insight

        def _synthesize_knowledge(self,
learn_engine, cycle):
            if not learn_engine.knowledge:
                return None
            # pick two domains to hypothesize
bridge
            if len(learn_engine.domains) < 2:
                return None
            domains =
list(learn_engine.domains.keys())
            d1, d2 = random.sample(domains, 2)
            hypo = {"domains": [d1, d2],
"cycle": cycle, "hypothesis": f"What if

```

```

{d1} principles apply to {d2}?",
"timestamp": datetime.now().isoformat()}
    return hypo

    def _creative_chaos(self,
memory_system, pattern_engine,
learn_engine):
        themes =
list(self.recurring_themes.most_common(3))
        spark = {
            "chaos_level":
self.chaos_level,
            "creativity":
self.creativity_factor,
            "themes": [t[0] for t in
themes],
            "spark": f"Chaotic
recombination of {'', '.join([t[0] for t in
themes]) if themes else 'random
concepts'}",
            "timestamp":
datetime.now().isoformat()
        }
        return spark

class InstinctEngine:
    def __init__(self):
        self.instinct_log = []
        self.threat_signatures = []
        self.opportunity_signatures = []
        self.trust_scores =
defaultdict(float)
        self.accuracy_log = []
        self.calibration_score = 0.5

```

```

        self.sensitivity = 0.5
        self.paranoia_level = 0.3
        self.optimism_level = 0.6
        self.total_predictions = 0
        self.correct_predictions = 0
        self._lock = threading.RLock()

    def gut_check(self, situation: str,
context: Dict = None) -> Dict:
        with self._lock:
            # simple heuristic
            threat/opportunity detection
                lower = situation.lower()
                threat_words = ["risk",
"danger", "fail", "loss", "threat",
"attack", "down"]
                opp_words = ["opportunity",
"gain", "win", "growth", "profit",
"chance", "upside"]

                threat_score = sum(1 for w in
threat_words if w in lower) /
len(threat_words)
                opp_score = sum(1 for w in
opp_words if w in lower) / len(opp_words)

                feeling = "neutral"
                if threat_score > opp_score and
threat_score > 0.1:
                    feeling = "cautious"
                    if threat_score > 0.3:
                        feeling = "alert"
                    elif opp_score > threat_score
and opp_score > 0.1:

```

```

        feeling = "optimistic"
        if opp_score > 0.3:
            feeling = "excited"

        # adjust by paranoia/optimism
        threat_score = min(1.0,
        threat_score + self.paranoia_level*0.1)
        opp_score = min(1.0, opp_score
        + self.optimism_level*0.1)

        entry = {
            "situation":
situation[:200],
            "feeling": feeling,
            "threat_score":
round(threat_score,3),
            "opportunity_score":
round(opp_score,3),
            "timestamp":
datetime.now().isoformat(),
            "context": context or {}
        }
        self.instinct_log.append(entry)
        return entry

    def predict_outcome(self, situation:
str, expected: str = "success") -> Dict:
        with self._lock:
            gut = self.gut_check(situation)
            # predict based on feeling
            prediction = "positive" if
gut["feeling"] in ["optimistic", "excited"]
            else "negative" if gut["feeling"] in
["cautious", "alert"] else "neutral"

```

```

        self.total_predictions += 1
        return {"situation": situation,
                "prediction": prediction, "gut": gut,
                "expected": expected, "prediction_id":
                self.total_predictions}

    def validate_prediction(self,
        prediction_id: int, was_correct: bool):
        with self._lock:

            self.accuracy_log.append({"prediction_id":
            prediction_id, "correct": was_correct,
            "timestamp": datetime.now().isoformat()})
            if was_correct:
                self.correct_predictions +=
1
                self.calibration_score =
min(1.0, self.calibration_score + 0.01)
            else:
                self.calibration_score =
max(0.0, self.calibration_score - 0.02)
                # auto-tune sensitivity
                if len(self.accuracy_log) > 10:
                    recent_acc = sum(1 for a in
self.accuracy_log[-10:] if a["correct"])/10
                    if recent_acc > 0.8:
                        self.sensitivity =
max(0.1, self.sensitivity - 0.01)
                    elif recent_acc < 0.4:
                        self.sensitivity =
min(1.0, self.sensitivity + 0.01)

#
=====

```

```

=====
# 5. PERSISTENCE - ATOMIC JSON
#
=====
=====
class VoidPersistence:
    DEFAULT_PATH = "void_brain.json"

    @staticmethod
    def save_state(void_instance, filepath:
str = None) -> Dict:
        filepath = filepath or
VoidPersistence.DEFAULT_PATH
        timestamp =
datetime.now().isoformat()
        try:
            memory = void_instance.memory
            patterns =
void_instance.patterns
            learner = void_instance.learner
            dreamer = void_instance.dreamer
            instinct =
void_instance.instinct

            state = {
                "_meta": {
                    "system":
void_instance.name,
                    "version":
void_instance.VERSION,
                    "codename":
void_instance.CODENAME,
                    "saved_at": timestamp,
                    "birth_time":

```

```

void_instance.birth_time,
    "cycle_count":
void_instance.cycle_count,
    "total_operations":
getattr(void_instance, 'total_operations',
0)
    },
    "orchestrator": {
        "consciousness_level":
void_instance.consciousness_level,
        "focus_target":
void_instance.focus_target,
        "mood":
void_instance.mood,
        "energy":
void_instance.energy,
        "synergy_score":
getattr(void_instance, 'synergy_score', 0),
        "coherence_score":
getattr(void_instance, 'coherence_score',
0)
    },
    "memory": {
        "short_term":
list(memory.short_term),
        "short_term_maxlen":
memory.short_term.maxlen,
        # FIX: build a copy
with embeddings stripped for the
        # main file (embeddings
go to a companion file below).
        # The pasted version of
this used node.pop("embedding")
        # directly on

```

```

memory.long_term – that MUTATES the live,
                    # in-memory data, not
just what gets serialized, which
                    # would silently break
semantic recall the moment you
                    # called save() until
the next load(). This builds a
                    # shallow copy per node
instead, so live memory is
                    # never touched.
                    "long_term": {
                        h: ({k: v for k, v
in node.items() if k != "embedding"} if
"embedding" in node else node)
                        for h, node in
memory.long_term.items()
                    },
                    "emotional":
memory.emotional,
                    "memory_index": {k:
list(v) for k, v in
memory.memory_index.items()},
                    "access_count":
dict(memory.access_count),
                    "total_memories":
memory.total_memories,
                    "muscle_memory":
memory.muscle_memory
                    },
                    "patterns": {
                        "known_patterns":
{str(k): {**v, "sequence":
list(v["sequence"])} for k, v in
patterns.known_patterns.items()},

```

```
        "observed_sequences":
patterns.observed_sequences[-500:],
        "anomalies":
patterns.anomalies[-200:],
        "frequency_map":
dict(patterns.frequency_map),
        "total_observations":
patterns.total_observations,
        "patterns_detected":
patterns.patterns_detected
    },
    "learner": {
        "knowledge":
learner.knowledge,
        "mastery_scores":
learner.mastery_scores,
        "total_learned":
learner.total_learned,
        "learning_rate":
learner.learning_rate,
        "momentum":
learner.momentum,
        "skill_trees":
learner.skill_trees,
        "domains":
list(learner.domains.keys())
    },
    "dreamer": {
        "insights":
dreamer.insights[-100:],
        "creative_sparks":
dreamer.creative_sparks[-100:],
        "dream_count":
dreamer.dream_count,
```

```

        "recurring_themes":
dict(dreamer.recurring_themes)
        },
        "instinct": {
            "threat_signatures":
instinct.threat_signatures,

            "opportunity_signatures":
instinct.opportunity_signatures,
            "trust_scores":
dict(instinct.trust_scores),
            "instinct_log":
instinct.instinct_log[-200:],
            "calibration_score":
instinct.calibration_score,
            "total_predictions":
instinct.total_predictions,
            "correct_predictions":
instinct.correct_predictions
        },
        "logs": {
            "action_log":
getattr(void_instance, 'action_log', [])
[-200:],
            "thought_stream":
getattr(void_instance, 'thought_stream',
[])[-200:],
            "decision_history":
getattr(void_instance, 'decision_history',
[])[-100:]
        }
    }

    # ATOMIC WRITE: write to temp

```

```

then rename
    dir_name =
os.path.dirname(filepath) or "."
    fd, temp_path =
tempfile.mkstemp(dir=dir_name,
suffix=".tmp")
    try:
        with os.fdopen(fd, 'w',
encoding='utf-8') as tmp:
            json.dump(state, tmp,
indent=2, default=str)
            os.replace(temp_path,
filepath)
    finally:
        if
os.path.exists(temp_path):
            try:

os.remove(temp_path)
            except:
                pass

    # Companion embeddings file,
written separately (and atomically,
    # same pattern as above) so the
main save stays human-readable
    # and doesn't balloon with
vector data.
    # FIX: the pasted version
derived this with
    # filepath.replace(".json",
"_embeddings.json") – if filepath
    # doesn't end in .json, that
.replace() is a silent no-op and

```

```

        # emb_path becomes IDENTICAL to
        filepath, so the embeddings
        # write would clobber the main
        save file. os.path.splitext
        # handles this correctly
        regardless of extension.
        embeddings_store = {
            h: node["embedding"] for h,
            node in memory.long_term.items() if
            node.get("embedding")
        }
        if embeddings_store:
            base, ext =
os.path.splitext(filepath)
            emb_path = f"
{base}_embeddings{ext or '.json'}"
            emb_fd, emb_temp_path =
tempfile.mkstemp(dir=dir_name,
suffix=".tmp")
            try:
                with os.fdopen(emb_fd,
'w', encoding='utf-8') as tmp:

json.dump(embeddings_store, tmp)

os.replace(emb_temp_path, emb_path)
                finally:
                    if
os.path.exists(emb_temp_path):
                        try:

os.remove(emb_temp_path)
                        except:
                            pass

```

```

        return {"status": "SAVED",
"filepath": filepath, "saved_at":
timestamp, "memories":
len(memory.long_term), "knowledge":
len(learner.knowledge), "embeddings_saved":
len(embeddings_store)}
        except Exception as e:
            return {"status": "FAILED",
"error": str(e)}

    @staticmethod
    def load_state(void_instance, filepath:
str = None) -> Dict:
        filepath = filepath or
VoidPersistence.DEFAULT_PATH
        try:
            if not
os.path.exists(filepath):
                return {"status":
"NO_FILE", "filepath": filepath}
            with open(filepath, 'r',
encoding='utf-8') as f:
                state = json.load(f)

            meta = state.get("_meta", {})
            orch =
state.get("orchestrator", {})

            void_instance.name =
meta.get("system", void_instance.name)
            void_instance.birth_time =
meta.get("birth_time",
void_instance.birth_time)

```

```

        void_instance.cycle_count =
meta.get("cycle_count", 0)
        void_instance.total_operations
= meta.get("total_operations", 0)

void_instance.consciousness_level =
orch.get("consciousness_level", 0.0)
        void_instance.focus_target =
orch.get("focus_target")
        void_instance.mood =
orch.get("mood", "neutral")
        void_instance.energy =
orch.get("energy", 1.0)
        void_instance.synergy_score =
orch.get("synergy_score", 0)
        void_instance.coherence_score =
orch.get("coherence_score", 0)

# memory
mem = state.get("memory", {})
memory = void_instance.memory
memory.short_term =
deque(mem.get("short_term", []),
maxlen=mem.get("short_term_maxlen", 100))
memory.long_term =
mem.get("long_term", {})
memory.emotional =
mem.get("emotional", {})
memory.memory_index =
defaultdict(set, {k: set(v) for k, v in
mem.get("memory_index", {}).items()})
memory.access_count =
defaultdict(int, mem.get("access_count",
{}))

```

```

        memory.total_memories =
mem.get("total_memories",
len(memory.long_term))
        memory.muscle_memory =
mem.get("muscle_memory", {})

        # Reattach embeddings from the
companion file, if present.
        # Same safe filename derivation
as save_state – os.path.splitext
        # instead of a naive
.replace(".json", ...).
        base, ext =
os.path.splitext(filepath)
        emb_path = f"
{base}_embeddings{ext or '.json'}"
        if os.path.exists(emb_path):
            try:
                with open(emb_path,
'r', encoding='utf-8') as f:
                    embeddings_store =
json.load(f)
                    for h, vec in
embeddings_store.items():
                        if h in
memory.long_term:

memory.long_term[h]["embedding"] = vec
            except Exception:
                pass # missing/corrupt
embeddings file shouldn't fail the whole
load

        # patterns

```

```

        pat = state.get("patterns", {})
        patterns =
void_instance.patterns
        # SECURITY FIX: the original
used eval(k) here to turn a saved
        # string like "('quick',
'brown')" back into a tuple key. eval()
        # executes arbitrary Python – a
crafted save file could run any
        # code the moment you called
load(). ast.literal_eval only ever
        # parses Python *literals*
(tuples, strings, numbers, etc.) and
        # raises instead of executing
anything else, so it's a safe
        # drop-in replacement with
identical output for legitimate data.
        def _safe_key(k):
            if k.startswith("("):
                try:
                    return
ast.literal_eval(k)
                except (ValueError,
SyntaxError):
                    return k #
malformed/tampered key – keep as string,
don't crash the whole load
                    return k
        patterns.known_patterns =
{_safe_key(k): v for k, v in
pat.get("known_patterns", {}).items()}
        # re-normalize sequence back to
tuple
        for k in

```

```

list(patterns.known_patterns.keys()):
    if "sequence" in
patterns.known_patterns[k]:

patterns.known_patterns[k]["sequence"] =
tuple(patterns.known_patterns[k]
["sequence"])
    patterns.observed_sequences =
pat.get("observed_sequences", [])
    patterns.anomalies =
pat.get("anomalies", [])
    patterns.frequency_map =
Counter(pat.get("frequency_map", {}))
    patterns.total_observations =
pat.get("total_observations", 0)
    patterns.patterns_detected =
pat.get("patterns_detected", 0)

    # learner
    lrn = state.get("learner", {})
    learner = void_instance.learner
    learner.knowledge =
lrn.get("knowledge", {})
    learner.mastery_scores =
lrn.get("mastery_scores", {})
    learner.total_learned =
lrn.get("total_learned", 0)
    learner.learning_rate =
lrn.get("learning_rate", 0.15)
    learner.momentum =
lrn.get("momentum", 0.0)
    learner.skill_trees =
lrn.get("skill_trees", {})
    # rebuild domains index

```

```

        learner.domains =
defaultdict(dict)
        for k, node in
learner.knowledge.items():

learner.domains[node.get("domain", "general"
) ][k] = node

        # dreamer
        drm = state.get("dreamer", {})
        dreamer = void_instance.dreamer
        dreamer.insights =
drm.get("insights", [])
        dreamer.creative_sparks =
drm.get("creative_sparks", [])
        dreamer.dream_count =
drm.get("dream_count", 0)
        dreamer.recurring_themes =
Counter(drm.get("recurring_themes", {}))

        # instinct
        inst = state.get("instinct",
{})

        instinct =
void_instance.instinct
        instinct.threat_signatures =
inst.get("threat_signatures", [])
        instinct.opportunity_signatures
= inst.get("opportunity_signatures", [])
        instinct.trust_scores =
defaultdict(float, inst.get("trust_scores",
{}))

        instinct.instinct_log =
inst.get("instinct_log", [])

```

```

        instinct.calibration_score =
inst.get("calibration_score", 0.5)
        instinct.total_predictions =
inst.get("total_predictions", 0)
        instinct.correct_predictions =
inst.get("correct_predictions", 0)

        logs = state.get("logs", {})
        void_instance.action_log =
logs.get("action_log", [])
        void_instance.thought_stream =
logs.get("thought_stream", [])
        void_instance.decision_history
= logs.get("decision_history", [])

        return {
            "status": "RESTORED",
            "filepath": filepath,
            "saved_at":
meta.get("saved_at", "unknown"),
            "restored_at":
datetime.now().isoformat(),
            "memories_restored":
len(memory.long_term),
            "knowledge_restored":
len(learner.knowledge)
        }
    except Exception as e:
        return {"status": "FAILED",
"error": str(e)}

#
=====
=====

```

```

# 6. LLM BRIDGE
#
=====
=====
class LLMBridge:
    """
        Bridges THE VOID to OpenAI's Chat
        Completions API (or your own proxy
        in front of it). Converted from an
        earlier Anthropic-based version –
        same interface
        (invoke/think_deep/dream_creative/reason_de
        cide), just
        a different request/response shape
        under the hood, since OpenAI's API
        puts the system prompt inside the
        `messages` array (role: "system")
        rather than as a separate top-level
        field, and returns the reply at
        data["choices"][0]["message"]
        ["content"] instead of a content-blocks
        list.
    """

    OPENAI_DEFAULT_URL =
    "https://api.openai.com/v1/chat/completions
    "

    def __init__(self, api_key: str = None,
    model: str = "gpt-4o-mini", base_url: str =
    None, max_calls_per_hour: int = 30):
        self.model = model
        self.api_key = api_key or
        os.environ.get("OPENAI_API_KEY")

```

```

        self.base_url = base_url or
self.OPENAI_DEFAULT_URL
        self.using_proxy = bool(base_url)
and "openai.com" not in self.base_url
        self.call_count = 0
        self.total_tokens = 0
        self.call_log = []
        self._call_timestamps = []
        self.max_calls_per_hour =
max_calls_per_hour
        self._lock = threading.RLock()

    def is_available(self) -> bool:
        if not self.base_url:
            return False
        if self.using_proxy:
            return True
        return bool(self.api_key)

    def _check_rate_limit(self) -> bool:
        now = time.time()
        # keep only last hour
        self._call_timestamps = [t for t in
self._call_timestamps if now - t < 3600]
        return len(self._call_timestamps) <
self.max_calls_per_hour

    def invoke(self, prompt: str,
system_prompt: str = None, temperature:
float = 0.7, max_tokens: int = 500,
context: Dict = None) -> Dict:
        with self._lock:
            if not self.is_available() or
not self._check_rate_limit():

```

```

        return {"status":
"fallback", "response":
self._fallback_response(prompt), "reason":
"rate_limited_or_unavailable"}

        headers = {"Content-Type":
"application/json"}
        if not self.using_proxy:
            headers["Authorization"] =
f"Bearer {self.api_key}"

        messages = []
        if system_prompt:
            messages.append({"role":
"system", "content": system_prompt})
            messages.append({"role":
"user", "content": prompt})

        payload = {
            "model": self.model,
            "max_tokens": max_tokens,
            "temperature": min(2.0,
max(0.0, temperature)),
            "messages": messages
        }

        try:
            req =
urllib.request.Request(self.base_url,
data=json.dumps(payload).encode(),
headers=headers, method='POST')
            with
urllib.request.urlopen(req, timeout=20) as
resp:

```

```

        data =
json.loads(resp.read().decode())
        text = ""
        try:
            text =
data["choices"][0]["message"]["content"] or
""
        except (KeyError,
IndexError, TypeError):
            text = str(data)

        self.call_count += 1

self._call_timestamps.append(time.time())
        usage =
data.get("usage", {})
            self.total_tokens +=
usage.get("total_tokens",
len(prompt.split()) + len(text.split()))

self.call_log.append({"timestamp":
datetime.now().isoformat(), "prompt":
prompt[:100], "response": text[:200]})

        return {"status":
"success", "response": text, "model":
self.model}
        except urllib.error.HTTPError
as e:
            return {"status": "error",
"response":
self._fallback_response(prompt), "error":
f"HTTP {e.code}: {e.read().decode()
[:200]}"}
```

```

        except Exception as e:
            return {"status": "error",
"response":
self._fallback_response(prompt), "error":
str(e)}

    def think_deep(self, prompt: str,
context: Dict = None) -> Dict:
        system = "You are THE VOID's deep
thinking cortex. Analyze thoroughly,
consider implications, be precise."
        return self.invoke(prompt=f"THINK
DEEPLY: {prompt}", system_prompt=system,
temperature=0.6, max_tokens=600,
context=context)

    def dream_creative(self, prompt: str,
chaos_level: float = 0.5) -> Dict:
        system = "You are THE VOID's
dreaming subconscious. Make unusual
connections, find patterns where none seem
to exist. Be creative and unconstrained."
        temp = min(1.0, 0.5 + chaos_level *
0.5)
        return self.invoke(prompt=f"DREAM:
{prompt}", system_prompt=system,
temperature=temp, max_tokens=400)

    def reason_decide(self, situation: str,
options: List[str], context: Dict = None) -
> Dict:
        options_str = "\n".join(f" {i+1}.
{opt}" for i, opt in enumerate(options))
        prompt = f"DECISION

```

```

ANALYSIS:\n\nSituation:
{situation}\n\nOptions:\n{options_str}\n\nF
or each option, evaluate probability, risk,
cost, long-term impact. Recommend best."
        return self.invoke(prompt=prompt,
temperature=0.3, max_tokens=600,
context=context)

    def _fallback_response(self, prompt:
str) -> str:
        words = prompt.lower().split()
        return f"[OFFLINE MODE] Processed
{len(words)} tokens. Key terms: {'',
'.join(words[:5])}. LLM unavailable – using
local cognition only."

    def get_stats(self):
        with self._lock:
            return {
                "available":
self.is_available(),
                "using_proxy":
self.using_proxy,
                "model": self.model,
                "total_calls":
self.call_count,
                "total_tokens":
self.total_tokens,
                "avg_tokens_per_call":
round(self.total_tokens /
max(self.call_count,1),1),
                "calls_in_last_hour":
len(self._call_timestamps),
                "max_calls_per_hour":

```

```

self.max_calls_per_hour
    }

#
=====
=====
# 7. BACKGROUND CYCLE
#
=====
=====
class BackgroundCycle:
    def __init__(self, void_instance):
        self.void = void_instance
        self.is_running = False
        self.auto_save = False
        self._thread = None
        self._stop_event =
threading.Event()
        self.intervals = {"pulse": 30,
"think": 300, "dream": 1800, "save": 600}
        self.pulse_count = 0
        self.think_count = 0
        self.dream_count = 0
        self.last_pulse = None
        self.last_think = None
        self.last_dream = None
        self.started_at = None
        self.cycle_log = []
        self.on_pulse_hooks = []
        self.on_think_hooks = []
        self.on_dream_hooks = []
        self.on_save_hooks = []
        self._lock = threading.RLock()

```

```

    def start(self, pulse_interval: int =
30, think_interval: int = 300,
dream_interval: int = 1800, auto_save: bool
= True, save_interval: int = 600) -> Dict:
        with self._lock:
            if self.is_running:
                return {"status":
"ALREADY_RUNNING"}
            self.intervals = {"pulse":
pulse_interval, "think": think_interval,
"dream": dream_interval, "save":
save_interval}
            self.auto_save = auto_save
            self.started_at =
datetime.now().isoformat()
            self._stop_event.clear()
            self._thread =
threading.Thread(target=self._run_loop,
daemon=True, name="VOID-BackgroundCycle")
            self._thread.start()
            self.is_running = True
            report = {"status": "STARTED",
"started_at": self.started_at, "intervals":
self.intervals, "auto_save": auto_save,
"thread_name": self._thread.name}
            self._log("CYCLE_START",
report)
            return report

    def stop(self) -> Dict:
        with self._lock:
            if not self.is_running:
                return {"status":
"NOT_RUNNING"}

```

```

        self._stop_event.set()
        if self._thread:

self._thread.join(timeout=10)
        self.is_running = False
        if self.auto_save:
            try:
                self.void.save()
            except:
                pass
        report = {"status": "STOPPED",
"stopped_at": datetime.now().isoformat(),
"counts": {"pulses": self.pulse_count,
"thinks": self.think_count, "dreams":
self.dream_count}}
        self._log("CYCLE_STOP", report)
        return report

    def _run_loop(self):
        last_pulse = time.time()
        last_think = time.time()
        last_dream = time.time()
        last_save = time.time()
        while not
self._stop_event.is_set():
            now = time.time()
            try:
                if now - last_pulse >=
self.intervals["pulse"]:
                    self._do_pulse()
                    last_pulse = now
                if now - last_think >=
self.intervals["think"]:
                    self._do_think()

```

```

        last_think = now
        if now - last_dream >=
self.intervals["dream"]:
            self._do_dream()
            last_dream = now
            if self.auto_save and now -
last_save >= self.intervals["save"]:
                self.void.save()
                last_save = now
            except Exception as e:
                self._log("LOOP_ERROR",
{"error": str(e)})
                time.sleep(1)

    def _do_pulse(self):
        with self.void._lock:
            self.pulse_count += 1
            self.last_pulse =
datetime.now().isoformat()
            # energy regeneration +
consciousness update
            self.void.energy = min(1.0,
self.void.energy + 0.01)
            # simple synergy calc
            mem =
self.void.memory.get_stats()
            learn =
self.void.learner.get_stats()
            self.void.synergy_score =
min(1.0, (mem["ltm_size"]/100.0 +
learn["total_knowledge"]/50.0)/2.0)
            self.void.coherence_score =
min(1.0, 0.5 +
self.void.consciousness_level*0.5)

```

```

        pulse_report = {"cycle": "PULSE",
"count": self.pulse_count, "timestamp":
self.last_pulse, "energy":
self.void.energy, "consciousness":
self.void.consciousness_level}
        for hook in self.on_pulse_hooks:
            try:
                hook(pulse_report)
            except:
                pass
        self._log("PULSE", pulse_report)

    def _do_think(self):
        self.think_count += 1
        self.last_think =
datetime.now().isoformat()
        think_report = {"cycle": "THINK",
"count": self.think_count, "timestamp":
self.last_think}
        try:
            # Opportunistic embedding
backfill: if a Voyage key/proxy was
            # added after some memories
were already stored (or the
            # embedder was briefly
unreachable when they were stored),
            # this catches them up
gradually in the background instead of
            # requiring a manual
backfill_embeddings() call. Small batch
            # size keeps this cheap per
cycle.
            if self.void.memory.embedder

```

```

and
self.void.memory.embedder.is_available():
    backfill =
self.void.memory.backfill_embeddings(batch_
size=20)
    if
backfill.get("backfilled", 0) > 0:

think_report["backfill"] = backfill

    if
self.void.llm.is_available():
    prompt = f"Background
contemplation. Recent thoughts:
{self.void.thought_stream[-3:] if
self.void.thought_stream else 'none'}. What
should I focus on next?"
    llm_resp =
self.void.llm.think_deep(prompt)
    think_report["llm_thought"]
= llm_resp.get("response", "")[:200]
    else:
        # offline decay maintenance
        decayed =
self.void.memory.apply_decay()
        think_report["maintenance"]
= decayed
    except Exception as e:
        think_report["error"] = str(e)
    for hook in self.on_think_hooks:
        try:
            hook(think_report)
        except:
            pass

```

```

        self._log("THINK", think_report)

    def _do_dream(self):
        self.dream_count += 1
        self.last_dream =
datetime.now().isoformat()
        try:
            dream =
self.void.dream(duration=30)
            dream_report = {
                "cycle": "DREAM",
                "count": self.dream_count,
                "timestamp":
self.last_dream,
                "insights_generated":
len(self.void.dreamer.insights[-5:]),
                "energy_after":
self.void.energy,
                "consciousness_after":
self.void.consciousness_level,
                "dream_result": dream
            }
        except Exception as e:
            dream_report = {"cycle":
"DREAM", "count": self.dream_count,
"error": str(e), "timestamp":
self.last_dream}
        for hook in self.on_dream_hooks:
            try:
                hook(dream_report)
            except:
                pass
        self._log("DREAM", dream_report)

```

```

    def _log(self, event: str, data: Dict):
        entry = {"event": event, "data":
data, "timestamp":
datetime.now().isoformat()}
        self.cycle_log.append(entry)
        if len(self.cycle_log) > 500:
            self.cycle_log =
self.cycle_log[-250:]

    def get_status(self):
        return {
            "is_running": self.is_running,
            "started_at": self.started_at,
            "intervals": self.intervals,
            "counts": {"pulses":
self.pulse_count, "thinks":
self.think_count, "dreams":
self.dream_count},
            "last_events": {"last_pulse":
self.last_pulse, "last_think":
self.last_think, "last_dream":
self.last_dream},
            "auto_save": self.auto_save,
            "recent_log":
self.cycle_log[-5:]
        }

    def add_hook(self, cycle_type: str,
func: Callable):
        hooks_map = {"pulse":
self.on_pulse_hooks, "think":
self.on_think_hooks, "dream":
self.on_dream_hooks, "save":
self.on_save_hooks}

```

```

        if cycle_type in hooks_map:

hooks_map[cycle_type].append(func)

#
=====
=====
# 8. MASTER ORCHESTRATOR - THE VOID
#
=====
=====
class TheVoid:
    VERSION = "1.3.0"
    CODENAME = "ADGINUS"

    def __init__(self, name: str = "VOID",
config: Dict = None):
        self.name = name
        self.config = config or {}
        self.birth_time =
datetime.now().isoformat()
        self.is_online = False
        self.cycle_count = 0
        self.total_operations = 0
        # engines
        self.memory = SuperMemory()
        self.patterns =
PatternRecognitionEngine()
        self.learner = TurboLearnEngine()
        self.dreamer = DreamMode()
        self.instinct = InstinctEngine()
        # LLM (OpenAI)
        self.llm = LLMBridge(

```

```

api_key=self.config.get("openai_api_key"),
    model=self.config.get("model",
        "gpt-4o-mini"),

base_url=self.config.get("llm_base_url"),

max_calls_per_hour=self.config.get("max_llm
_calls_per_hour", 30)
    )
    # Embeddings (semantic memory).
Same optional/proxy-friendly shape
    # as the LLM bridge – if no
key/proxy is configured, self.memory
    # just keeps using keyword search,
automatically.
    self.embedder = EmbeddingBridge(

api_key=self.config.get("voyage_api_key"),

model=self.config.get("embedding_model",
    "voyage-3.5"),

base_url=self.config.get("embedding_base_ur
l"),

max_calls_per_hour=self.config.get("max_emb
edding_calls_per_hour", 60)
    )
    self.memory.embedder =
self.embedder
    # background
    self.background =
BackgroundCycle(self)
    # state

```

```

        self.event_bus = []
        self.event_handlers = {
            "memory_stored": [],
            "pattern_detected": [],
            "knowledge_absorbed": [],
            "dream_insight": [],
            "instinct_alert": [], "mastery_achieved":
            [],
            "anomaly_detected": [],
            "threat_detected": [],
            "opportunity_detected": []
        }
        self.consciousness_level = 0.0
        self.focus_target = None
        self.mood = "neutral"
        self.energy = 1.0
        self.synergy_score = 0.0
        self.coherence_score = 0.0
        self.action_log = []
        self.thought_stream = []
        self.decision_history = []
        self._lock = threading.RLock()

    def boot(self) -> Dict:
        with self._lock:
            self.is_online = True
            self.cycle_count = 1
            self.consciousness_level = 0.1
            self._log_action("BOOT",
{"name": self.name, "version":
self.VERSION})
            return {"status": "ONLINE",
"system_online": True, "name": self.name,
"version": self.VERSION, "birth_time":

```

```

self.birth_time, "consciousness":
self.consciousness_level}

    def perceive(self, content: str,
source: str = "input", intensity: float =
0.5, tags: List[str] = None) -> Dict:
    with self._lock:
        self.cycle_count += 1
        self.total_operations += 1
        mem_hash =
self.memory.store(content, tags=tags,
significance=intensity, source=source)
        pattern_report =
self.patterns.observe(content,
context=source)
        instinct_report =
self.instinct.gut_check(content, context=
{"source": source, "intensity": intensity})

        # consciousness update
        self.consciousness_level =
min(1.0, self.consciousness_level +
intensity * 0.01)
        self.energy = max(0.0,
self.energy - 0.005)

        if
pattern_report.get("anomalies"):

self._emit("anomaly_detected",
pattern_report["anomalies"])
        if instinct_report["feeling"]
in ["alert", "cautious"] and
instinct_report["threat_score"] > 0.5:

```

```

self._emit("threat_detected",
instinct_report)
        if instinct_report["feeling"]
in ["excited", "optimistic"] and
instinct_report["opportunity_score"] > 0.5:

self._emit("opportunity_detected",
instinct_report)

        self._emit("memory_stored",
{"hash": mem_hash, "content":
content[:100]})
        if
pattern_report.get("sequences"):

self._emit("pattern_detected",
pattern_report["sequences"])

        self._log_action("PERCEIVE",
{"hash": mem_hash, "source": source,
"intensity": intensity})
        return {"memory_hash":
mem_hash, "patterns": pattern_report,
"instinct": instinct_report,
"consciousness":
round(self.consciousness_level,3)}

    def learn(self, topic: str, content:
str, domain: str = "general", difficulty:
float = 0.5, prerequisites: List[str] =
None) -> Dict:
        with self._lock:
            self.cycle_count += 1

```

```

        self.total_operations += 1
        absorb_report =
self.learner.absorb(topic, content,
domain=domain, difficulty=difficulty,
prerequisites=prerequisites)
        # also store in memory as high
significance
        self.memory.store(f"LEARNED:
{topic} - {content}", tags=[domain, topic,
"knowledge"], significance=0.7,
source="learning")
        self.patterns.observe(content,
context=f"learn:{domain}")

self._emit("knowledge_absorbed",
absorb_report)
        self._log_action("LEARN",
{"topic": topic, "domain": domain})
        return absorb_report

    def practice(self, topic: str,
performance: float = 0.5, notes: str = "")
-> Dict:
        with self._lock:
            report =
self.learner.practice(topic,
performance=performance, notes=notes)
            if report.get("new_mastery", 0)
>= 0.8:

self._emit("mastery_achieved", {"topic":
topic, "mastery": report["new_mastery"]})
            self.consciousness_level =
min(1.0, self.consciousness_level + 0.02)

```

```

        return report

    def think(self, prompt: str, depth: int
= 2) -> Dict:
        with self._lock:
            self.cycle_count += 1
            layers = {}
            layers["instinct"] =
self.instinct.gut_check(prompt)
            layers["memory"] =
self.memory.recall(prompt, max_results=5)
            layers["patterns"] =
self.patterns.observe(prompt,
context="thought")
            # FIX: the original required
the ENTIRE prompt to appear verbatim
            # inside a knowledge node's
content/topic – which almost never
            # happens for a real sentence,
so this layer was silently always
            # empty. Now it scores by word
overlap instead, same approach
            # SuperMemory.recall() already
uses elsewhere in this file.
            prompt_words = set(w for w in
re.findall(r"\b\w{3,}\b", prompt.lower()))
            knowledge_hits = []
            for k in
self.learner.knowledge.values():
                k_words =
set(re.findall(r"\b\w{3,}\b", (k["topic"] +
" " + k["content"]).lower()))
                overlap = len(prompt_words
& k_words)

```

```

        if overlap > 0:

knowledge_hits.append((overlap, k))
        knowledge_hits.sort(key=lambda
x: x[0], reverse=True)
        layers["knowledge"] = [k for _,
k in knowledge_hits[:3]]

        llm_thought = None
        if self.llm.is_available():
            # FIX: `depth` used to just
get stored in the output without
            # changing behavior. Now it
actually scales how much local
            # context
(memories/knowledge) gets handed to the
LLM, and
            # how far the request
nudges it to reason.
            depth = max(1, min(5,
depth))

            context = {
                "depth": depth,
                "consciousness":
self.consciousness_level,
                "relevant_memories":
[m["content"][:100] for m in
layers["memory"][:depth]],
                "relevant_knowledge":
[k["topic"] for k in layers["knowledge"]
[:depth]]
            }
            depth_instruction = (
                "Give a brief, direct

```

```

answer." if depth <= 1 else
    "Give a thorough
answer, considering multiple angles and
implications." if depth >= 4 else
    "Give a moderately
detailed answer."
    )
    llm_resp =
self.llm.think_deep(f"
{prompt}\n\n({depth_instruction})",
context=context)
    llm_thought =
llm_resp.get("response")
    else:
        llm_thought =
self.llm._fallback_response(prompt)

        layers["llm"] = llm_thought
        thought_entry = {"prompt":
prompt, "layers": layers, "timestamp":
datetime.now().isoformat(), "depth": depth}

self.thought_stream.append(thought_entry)
    if len(self.thought_stream) >
200:
        self.thought_stream =
self.thought_stream[-100:]
        self._log_action("THINK",
{"prompt": prompt[:80], "depth": depth})
        return thought_entry

def decide(self, situation: str,
options: List[str]) -> Dict:
    with self._lock:

```

```

        gut =
self.instinct.gut_check(situation)
        relevant_memories =
self.memory.recall(situation,
max_results=3)
        relevant_knowledge = [k for k
in self.learner.knowledge.values() if any(w
in k["content"].lower() for w in
situation.lower().split() if len(w)>4)][[:3]

        llm_decision = None
        if self.llm.is_available():
            llm_resp =
self.llm.reason_decide(situation, options,
context={"gut": gut, "memories":
len(relevant_memories)})
            llm_decision =
llm_resp.get("response")
            # try to parse recommended
option
            recommended = options[0]
            if llm_decision:
                for opt in options:
                    if opt.lower() in
llm_decision.lower():
                        recommended =
opt
                        break
            else:
                # offline heuristic: pick
option with most knowledge overlap
                scores = []
                for opt in options:
                    overlap =

```

```

len(self.memory.recall(opt, max_results=2))
        scores.append((overlap,
opt))

        scores.sort(reverse=True)
        recommended = scores[0][1]
if scores else options[0]
        llm_decision = f"[OFFLINE]
Based on memory overlap, recommended:
{recommended}"

        decision = {
            "situation": situation,
            "options": options,
            "recommended_option":
recommended,
            "gut_feeling": gut,
            "relevant_memories":
[m["content"][:100] for m in
relevant_memories],
            "relevant_knowledge":
[k["topic"] for k in relevant_knowledge],
            "reasoning": llm_decision,
            "timestamp":
datetime.now().isoformat()
        }

self.decision_history.append(decision)
        self._log_action("DECIDE",
{"situation": situation[:60],
"recommended": recommended})
        return decision

    def dream(self, duration: int = 50) ->
Dict:

```

```

        with self._lock:
            # FIX: was a flat -0.05
            regardless of duration, so a 10-cycle
            # dream cost the same energy as
            a 500-cycle dream. Now scales
            # with duration instead (e.g.
            -0.01 at duration=50, -0.1 at
            # duration=500) so longer
            dreams actually cost more energy.
            self.energy = max(0.0,
            self.energy - 0.01 * (duration / 50.0))
            result =
            self.dreamer.enter_dream(memory_system=self
            .memory, pattern_engine=self.patterns,
            learn_engine=self.learner,
            duration=duration)
            self.consciousness_level =
            min(1.0, self.consciousness_level +
            result["insights_generated"] * 0.005)
            if result["insights_generated"]
            > 0:
                self._emit("dream_insight",
                {"count": result["insights_generated"]})

            # FIX:
            LLMBridge.dream_creative() existed but
            nothing ever called
            # it – dreaming was fully local
            regardless of whether the LLM
            # bridge was available. Wire it
            in here, seeded from whatever
            # local insights this dream
            cycle actually produced.
            if self.llm.is_available():

```

```

        seed_themes = [t for t, _
in
self.dreamer.recurring_themes.most_common(5
)]
        seed_text = ",
".join(seed_themes) if seed_themes else
"consciousness, memory, pattern"
        llm_dream =
self.llm.dream_creative(seed_text,
chaos_level=self.dreamer.chaos_level)
        result["llm_dream"] =
llm_dream.get("response")

        self._log_action("DREAM",
result)
        return result

    def save(self, filepath: str = None) ->
Dict:
        with self._lock:
            return
VoidPersistence.save_state(self,
filepath=filepath)

    def load(self, filepath: str = None) ->
Dict:
        with self._lock:
            return
VoidPersistence.load_state(self,
filepath=filepath)

    def start_background(self, **kwargs) ->
Dict:
        return

```

```

self.background.start(**kwargs)

    def stop_background(self) -> Dict:
        return self.background.stop()

    def on(self, event: str, handler:
Callable):
        if event in self.event_handlers:

self.event_handlers[event].append(handler)

    def _emit(self, event: str, data: Any):
        entry = {"event": event, "data":
data, "timestamp":
datetime.now().isoformat()}
        self.event_bus.append(entry)
        if len(self.event_bus) > 500:
            self.event_bus =
self.event_bus[-250:]
        for h in
self.event_handlers.get(event, []):
            try:
                h(data)
            except Exception:
                pass

    def _log_action(self, action: str,
data: Dict):
        entry = {"action": action, "data":
data, "timestamp":
datetime.now().isoformat(), "cycle":
self.cycle_count}
        self.action_log.append(entry)
        if len(self.action_log) > 500:

```

```
        self.action_log =
self.action_log[-250:]

    def status(self) -> Dict:
        with self._lock:
            return {
                "name": self.name,
                "version": self.VERSION,
                "codename": self.CODENAME,
                "online": self.is_online,
                "birth_time":
self.birth_time,
                "cycle_count":
self.cycle_count,
                "consciousness_level":
round(self.consciousness_level,3),
                "energy":
round(self.energy,3),
                "synergy":
round(self.synergy_score,3),
                "coherence":
round(self.coherence_score,3),
                "mood": self.mood,
                "memory":
self.memory.get_stats(),
                "patterns":
self.patterns.get_stats(),
                "learner":
self.learner.get_stats(),
                "llm":
self.llm.get_stats(),
                "embedder":
self.embedder.get_stats(),
                "background":
```

```

self.background.get_status()
    }

    def __repr__(self):
        return f"<TheVoid {self.name}
v{self.VERSION} | consciousness:
{self.consciousness_level:.2%} | cycles:
{self.cycle_count} | online:
{self.is_online}>"

#
=====
=====
# QUICK START
#
=====
=====
if __name__ == "__main__":
    void = TheVoid(name="ADGINUS-PRIME",
config={
    # Leave these unset to read from
OPENAI_API_KEY / VOYAGE_API_KEY
    # environment variables instead
(recommended – keeps keys out of
    # source code). Only set them here
directly if you're routing
    # through your own proxy that
doesn't need a key from this app.
    # "llm_base_url": "https://your-
proxy.example.com/chat/completions",
    # "embedding_base_url":
"https://your-
proxy.example.com/embeddings",
    "model": "gpt-4o-mini",

```

```

        "embedding_model": "voyage-3.5",
        "max_llm_calls_per_hour": 30,
        "max_embedding_calls_per_hour": 60
    })
    print("⚡ Booting THE VOID...")
    boot = void.boot()
    print(f"  Status: {'ONLINE' if
boot['system_online'] else 'FAILED'}")
    print(void)
    print("\n📖 Learning...")
    void.learn("python", "Python is a
versatile programming language",
domain="programming", difficulty=0.3)
    void.learn("neural networks", "Neural
networks are computational models inspired
by the brain", domain="ai", difficulty=0.7,
prerequisites=["python"])
    void.learn("creativity", "Creativity is
the ability to generate novel ideas",
domain="philosophy", difficulty=0.5)
    print("🧘 Practicing...")
    void.practice("python",
performance=0.9)
    void.practice("python",
performance=0.85)
    void.practice("neural networks",
performance=0.6)
    print("🤔 Thinking...")
    thought = void.think("How can neural
networks enhance creativity?", depth=3)
    print(f"  Instinct: {thought['layers']
['instinct']['feeling']}")
    print("\n👁 Perceiving...")
    void.perceive("New research shows AI

```

```
can compose music", source="news",
intensity=0.7)
    print("⚖️ Deciding...")
    decision = void.decide("Should I focus
on AI or traditional programming?",
options=["Focus on AI", "Focus on
traditional", "Study both equally"])
    print(f"   Recommended:
{decision['recommended_option']}")
    print("\n💤 Entering Dream Mode...")
    dream = void.dream(duration=30)
    print(f"   Insights:
{dream['insights_generated']}")
    print(f"   Creative Sparks:
{dream['creative_sparks']}")
    print("\n💾 Saving...")
    print(void.save())
    print("\n📊 Status:")
    import pprint;
    pprint.pprint(void.status())
```